ARP Spoofing Virtual Lab — Docker Implementation Guide

Experiment: Demonstrate ARP Spoofing and how incorrect ARP mappings affect communication
Distribution model: `git clone` → `docker compose up` → open `frontend/index.html` — runs identically on any teammate's or evaluator's machine, no manual dependency install.

---

0. GitHub repository setup (do this before anyone writes code)

The college's integration spec expects **one shared repository** with a branch per group (`main`, `group-EXP-1`, `group-EXP-2`, ...) — ask the integration/compilation team for the repo URL and your confirmed Experiment ID.

A) If the shared repo already exists:

```
git clone <shared-repo-url>
cd <repo-name>
git checkout -b group-EXP-2          # use your real confirmed EXP ID
mkdir -p experiments/arp-spoofing
git push -u origin group-EXP-2
```

B) If it doesn't exist yet (build standalone now, your PR becomes the merge point later):

```
mkdir arp-spoofing-lab && cd arp-spoofing-lab
git init
# create the folder structure from Section 4
git add . && git commit -m "Initial ARP spoofing lab skeleton"
gh repo create arp-spoofing-lab --private --source=. --push
# or: create the repo on github.com, then
# git remote add origin <url> && git push -u origin main
```

First commit, before any module work starts: create `docs/CONTRACT.md` with the exact contents of Section 3 below, and commit it on its own. This is the one file every teammate reads before writing a line of module code — it's what lets all 4 modules be built in parallel without waiting on each other.

Per-module branches, cut from `group-EXP-2` (or `main` if standalone), one per teammate:

```
git checkout -b module/network-infra      # Member 1
git checkout -b module/backend-api        # Member 2
git checkout -b module/frontend           # Member 3
git checkout -b module/content-docs       # Member 4
```

Each person opens a PR back into `group-EXP-2` once their module's acceptance test (listed at the end of their prompt in Section 6) passes. Because ownership is split by top-level folder (`nodes/`, `backend/`, `frontend/`, `content/`), merge conflicts should be rare — the only shared file anyone edits is `docker-compose.yml`, which stays with Module 1.

The flow for each teammate (only one person does A or B above):

1. One person does the repo setup (A or B) — folder structure, `lab.yaml`, `docs/CONTRACT.md` — commits and pushes `group-EXP-2`.
2. Everyone else **clones that same repo** (not a separate one each):

   ```
   git clone <repo-url>
   cd <repo-name>
   git checkout group-EXP-2
   ```

3. Each person checks out their own module branch off that:

   ```
   git checkout -b module/backend-api   # example
   ```

4. In Antigravity, open that checked-out repo (so the agent can see `docs/CONTRACT.md`, `lab.yaml`, and the folder structure already on disk), and give it **both**: the contents of `docs/CONTRACT.md` and your module's prompt from Section 6. Pointing it at the real contract file — not just pasting the prompt alone — is what keeps the four modules' file paths and endpoint names consistent.
5. Once your module passes its acceptance test, commit, push your branch, open a PR into `group-EXP-2`.

Root `.gitignore`:

```
__pycache__/
*.pyc
.env
.DS_Store
docker-compose.override.yml
```

---

1. Architecture

```
Browser (frontend/)
   │  REST + WebSocket
   ▼
Control API (backend/, FastAPI)
   │  Docker SDK (start/stop/exec containers)
   ▼
Isolated Docker network "arp-lab-net" (10.10.10.0/24)
   ├── gateway   10.10.10.1   (normal router behavior)
   ├── victim    10.10.10.10  (sends traffic, holds ARP cache)
   └── attacker  10.10.10.66  (scapy-based ARP poisoning + sniffing)
```

The **backend is the only thing that talks to Docker.** The frontend never touches containers directly — it only calls the backend's REST endpoints and listens on its WebSocket. This is what lets 4 people build in parallel: everyone codes against one shared contract (Section 3) instead of each other's internals.

2. Prerequisites (same for every teammate)

- Docker Desktop (Windows/Mac) or Docker Engine + Compose plugin (Linux)
- Git
- Python 3.11+ only needed if someone wants to run the backend outside Docker for debugging
- No Node.js required — frontend is plain HTML/CSS/JS per the college's "Required Libraries: None" convention

---

3. Shared contract (read this before writing any code)

All 4 modules must agree on this. Copy this section verbatim into `docs/CONTRACT.md` as the repo's first commit (see Section 0).

REST (backend, port 8000):

Method	Path	Body	Purpose
POST	`/api/lab/start`	—	Build + start all 3 containers
POST	`/api/lab/stop`	—	Stop containers, keep images
POST	`/api/lab/reset`	—	Stop, remove, rebuild clean state
POST	`/api/attack/start`	`{"mode":"mitm"|"dos"}`	Run ARP poison script in attacker container
POST	`/api/attack/stop`	—	Restore correct ARP mappings
POST	`/api/mitigation/toggle`	`{"enabled": true|false}`	Apply static ARP entries on victim/gateway
GET	`/api/state`	—	Current lab status + both ARP tables

WebSocket `ws://localhost:8000/ws/logs` — backend pushes JSON events, frontend only renders them:

```
{"type":"arp_update","node":"victim","ip":"10.10.10.1","mac":"<claimed-mac>","ts":16
{"type":"packet","from":"victim","to":"gateway","via":"attacker","encrypted":false,"
{"type":"attack_state","active":true,"mode":"mitm","ts":169...}
{"type":"mitigation_state","enabled":true,"ts":169...}
{"type":"log","level":"info","message":"...","ts":169...}
```

Everyone builds against this contract from day one; nobody waits for someone else's code to exist.

---

4. Repository structure

```
experiments/arp-spoofing/
├── lab.yaml
├── README.md                    ← Module 4
├── docker-compose.yml           ← Module 1
├── docs/CONTRACT.md
├── nodes/                       ← Module 1
│   ├── gateway/Dockerfile
│   ├── victim/Dockerfile
│   └── attacker/
│       ├── Dockerfile
│       └── arp_spoof.py
├── backend/                     ← Module 2
│   ├── app.py
│   ├── docker_control.py
│   ├── requirements.txt
│   └── Dockerfile
├── frontend/                    ← Module 3
│   ├── index.html
│   ├── style.css
│   └── script.js
├── content/                     ← Module 4
│   ├── theory.md
│   ├── quiz.json
│   └── references.md
└── tests/                       ← Module 4
```

5. `lab.yaml` manifest

```yaml
id: EXP02            # confirm with faculty/integration team
name: ARP Spoofing
mode: NETWORK_LAB
services: [gateway, victim, attacker]
network:
  isolated: true
  subnet: 10.10.10.0/24
limits:
  memory: 256m
  cpu: 0.5
```

---

6. Module breakdown (4 members)

Module 1 — Network Infrastructure

Owns: `docker-compose.yml`, `nodes/*`, the isolated network, the real ARP poisoning + legitimate traffic scripts. **Deliverable:** `docker compose up` brings up 3 containers on `arp-lab-net`; victim can ping gateway normally; a script inside `attacker` can poison both caches using scapy; everything is resource-limited and cannot reach any network outside `arp-lab-net`.

```
PROMPT FOR CLAUDE OPUS (ANTIGRAVITY) — MODULE 1

Context: I'm building a Docker-based cybersecurity teaching lab (ARP Spoofing demo)
inside experiments/arp-spoofing/. You own the network layer only — no frontend,
no backend/API code.

Task: Create:
1. docker-compose.yml defining 3 services on an isolated bridge network
   "arp-lab-net" (subnet 10.10.10.0/24, internal: true):
   - gateway  (10.10.10.1)  - lightweight Linux image (alpine) acting as router,
     IP forwarding enabled, no internet access
   - victim   (10.10.10.10) - alpine + python3 + scapy, periodically sends
     ICMP/HTTP-like requests to gateway
   - attacker (10.10.10.66) - alpine + python3 + scapy + net-tools
   Set mem_limit: 256m and cpus: 0.5 on every service. Do NOT publish any ports
   to the host except what's strictly needed for debugging.
2. nodes/attacker/arp_spoof.py: a scapy script with two functions:
   - start_poison(victim_ip, gateway_ip, mode) : sends forged ARP replies to
     victim (claiming to be gateway) and to gateway (claiming to be victim),
     repeating every 2s. mode="mitm" also enables IP forwarding + packet
     logging inside the container; mode="dos" drops forwarded packets instead.
   - stop_poison(): sends correct ARP replies to restore both caches and
     disables forwarding/dropping.
   Make it runnable as a long-lived process controllable by sending it
   commands over stdin or a tiny local socket (your choice) — the backend team
   (Module 2) will call this via `docker exec`, so document the exact command
   they should run for start/stop in nodes/attacker/README.md.
3. nodes/victim and nodes/gateway Dockerfiles: minimal alpine images with
   python3, scapy, iproute2, arping, tcpdump installed.

Constraints: no host-shell access from inside containers beyond their own
filesystem; nothing in this lab may reach the internet or the host LAN;
everything must clean up fully on `docker compose down -v`.

Acceptance test: `docker compose up -d`, then from victim container,
`ping -c3 10.10.10.1` succeeds; after running start_poison, victim's ARP
table shows the attacker's MAC for 10.10.10.1.
```

Module 2 — Backend / Control API

Owns: `backend/*`. Translates the REST+WebSocket contract (Section 3) into Docker actions. **Deliverable:** FastAPI service that can start/stop/reset the lab, trigger `arp_spoof.py` inside the attacker container via `docker exec`, tail its output, and stream parsed events to any connected frontend over WebSocket.

```
PROMPT FOR CLAUDE OPUS (ANTIGRAVITY) — MODULE 2
```

```
Context: Same ARP Spoofing Docker lab. The network layer (gateway, victim,
attacker containers + docker-compose.yml) is owned by a teammate and already
defines services named "gateway", "victim", "attacker" on network
"arp-lab-net". The attacker container has a script nodes/attacker/arp_spoof.py
with start_poison()/stop_poison() functions — assume it's controllable via a
documented `docker exec` command (check nodes/attacker/README.md once it
exists; for now assume `docker exec attacker python3 arp_spoof.py start
--mode mitm` and `... stop`).

Task: Build backend/app.py as a FastAPI app that implements exactly this
contract:

REST:
  POST /api/lab/start    -> `docker compose up -d`, wait for containers healthy
  POST /api/lab/stop     -> `docker compose stop`
  POST /api/lab/reset    -> `docker compose down -v && docker compose up -d`
  POST /api/attack/start {"mode": "mitm"|"dos"} -> docker exec into attacker,
       run arp_spoof.py start --mode <mode>
  POST /api/attack/stop  -> docker exec arp_spoof.py stop
  POST /api/mitigation/toggle {"enabled": bool} -> docker exec into victim
       and gateway to add/remove a static ARP entry for each other's real MAC
  GET  /api/state        -> returns lab status + current ARP table of victim
       and gateway (parsed from `docker exec <node> ip neigh` or `arp -a`)

WebSocket /ws/logs: broadcast JSON events to all connected clients matching
this schema exactly (reuse this, don't invent new fields):
  {"type":"arp_update","node":...,"ip":...,"mac":...,"ts":...}
  {"type":"attack_state","active":bool,"mode":...,"ts":...}
  {"type":"mitigation_state","enabled":bool,"ts":...}
  {"type":"log","level":"info"|"warn"|"error","message":...,"ts":...}
Poll each node's ARP table every 2s while the lab is running and emit
arp_update events on any change.

Use the `docker` Python SDK (not raw subprocess where avoidable) to control
containers. Wrap every docker call in try/except and emit a {"type":"log",
"level":"error",...} event on failure instead of crashing.

Also write backend/requirements.txt and backend/Dockerfile (python:3.11-slim
+ the docker CLI so it can control sibling containers via the host Docker
socket — document in README that docker-compose.yml must mount
/var/run/docker.sock into this service).

Acceptance test: starting the backend, calling POST /api/lab/start, then
POST /api/attack/start {"mode":"mitm"}, produces at least one arp_update
WebSocket event showing the victim's cache entry for the gateway IP now
pointing at the attacker's MAC.
```

Module 3 — Frontend

Owns: `frontend/*`. The 8 required sections, matching the common theme, driven live by the backend's WebSocket. **Deliverable:** one static page (`index.html + style.css + script.js` , no build step, no libraries — matches the SQL Injection module's "Required Libraries: None") with tabs: Aim, Objective, Theory, Procedure, Simulation, Assignment, References, Feedback

tabs: Aim, Objective, Theory, Procedure, Simulation, Assignment, References, Feedback.

```
PROMPT FOR CLAUDE OPUS (ANTIGRAVITY) — MODULE 3

Context: Same ARP Spoofing Docker lab. A backend service (owned by a
teammate) will run at http://localhost:8000 exposing this REST+WebSocket
contract — build against it exactly, do not invent different endpoint names:

REST: POST /api/lab/start, POST /api/lab/stop, POST /api/lab/reset,
POST /api/attack/start {"mode":"mitm"|"dos"}, POST /api/attack/stop,
POST /api/mitigation/toggle {"enabled":bool}, GET /api/state
WebSocket: ws://localhost:8000/ws/logs, messages are JSON objects with a
"type" field: "arp_update", "attack_state", "mitigation_state", "log".

Task: Build frontend/index.html, style.css, script.js as a single-page app
(no frameworks, no build tools — plain HTML/CSS/JS only) with these tabs:

1. Aim / 2. Objective / 3. Theory (ARP protocol, request/reply, ARP cache,
no authentication in ARP, gratuitous ARP, MITM vs DoS, defenses: static ARP,
Dynamic ARP Inspection) / 4. Procedure (numbered steps matching the buttons
below exactly) /
5. Simulation — the live dashboard:
   - A simple SVG/canvas diagram of 3 nodes (Victim, Gateway, Attacker) with
     their IP and current MAC-for-each-other shown as labelled arrows;
     redraw the arrows when an "arp_update" event arrives, and color an
     arrow red if it's pointing at the attacker's MAC instead of the real
     owner's.
   - Buttons: Start Lab, Stop Lab, Reset Lab, Start Attack (dropdown MITM/
     DoS), Stop Attack, Toggle Mitigation — each calls the matching REST
     endpoint with fetch().
   - A scrolling activity log panel rendering every "log" and state-change
     WebSocket event with a timestamp.
   - A status badge: "Stopped" / "Running — Protected" / "Running —
     Vulnerable" / "Under Attack", derived from attack_state + mitigation_state
     events.
6. Assignment — render content/quiz.json (array of {question, options,
   correctIndex, explanation}) as a one-question-at-a-time quiz with instant
   feedback and a final score; if content/quiz.json doesn't exist yet, stub
   it with 3 placeholder questions so the UI is testable standalone.
7. References — render content/references.md as a plain list.
8. Feedback — a simple form (name optional, rating 1–5, comment) that just
   shows a thank-you message on submit (no backend storage needed).

Style: define all colors/fonts as CSS custom properties at the top of
style.css under a clearly commented "THEME — integration team will override
these" block, so the integration team can restyle without touching markup.
Keep the whole thing responsive (flex/grid, no fixed pixel widths on the
main layout).

Acceptance test: opening index.html directly in a browser (file://) renders
all 8 tabs without errors even if the backend isn't running; with the
backend running, clicking Start Lab → Start Attack visibly turns an arrow
red within ~2 seconds.
```

Module 4 — Content, Quiz, Docs & QA

Owns: `content/*`, `README.md`, `tests/*`, the integration checklist, cross-platform setup instructions. **Deliverable:** theory writeup, 8–10 question quiz, references, the integration-team README (exact fields their spec requires), a test checklist, and a Windows/Mac/Linux "how to run this" guide for anyone without Docker experience.

```
PROMPT FOR CLAUDE OPUS (ANTIGRAVITY) — MODULE 4

Context: Same ARP Spoofing Docker lab (experiments/arp-spoofing/), built by 3
teammates covering network containers, a FastAPI backend, and a static
frontend. You own content and documentation only.

Task:
1. content/theory.md — concise theory covering: OSI layer 2/3 and MAC vs IP,
   how ARP request/reply works, the ARP cache, why ARP has no authentication,
   gratuitous ARP, how poisoning redirects traffic, MITM vs DoS outcomes,
   and defenses (static ARP entries, Dynamic ARP Inspection, port security,
   why HTTPS/VPN limits damage even under a successful MITM).
2. content/quiz.json — 8–10 multiple-choice questions as an array of objects
   {"question": str, "options": [str,...], "correctIndex": int,
   "explanation": str}. Mix recall questions (e.g. which protocol resolves
   IP to MAC) with scenario questions describing a specific poisoned ARP
   table and asking what happened or what it enables.
3. content/references.md — list: RFC 826; Kurose & Ross "Computer
   Networking: A Top-Down Approach" (ARP section); Forouzan "Data
   Communications and Networking" (ARP chapter); MITRE ATT&CK T1557.002
   (ARP Cache Poisoning); Cisco Dynamic ARP Inspection documentation;
   Wireshark wiki ARP page. One line each, title + what it covers.
4. README.md at experiments/arp-spoofing/ root, following this exact
   integration-team template (fill every field):
   Experiment ID / Experiment Name / Folder / Entry File / Navigation Title
   / Short Description / Required Libraries / Input / Output / Expected
   Navigation Link / Learning Outcome / Virtual Lab Components (numbered
   list) — plus a "How to run" section: `docker compose up -d --build` then
   open frontend/index.html, and a "How to stop / reset" section.
5. A plain-language setup guide, docs/SETUP.md, for Windows, macOS and
   Linux: installing Docker Desktop/Engine, cloning the repo, running the
   lab, and the 3 most likely errors (Docker not running, port already in
   use, antivirus blocking the isolated network) with fixes.
6. tests/CHECKLIST.md covering: fresh install works with no manual deps;
   normal traffic works before any attack; attack reproduces the expected
   poisoned ARP state; mitigation blocks/reduces it; reset returns to a
   clean state; lab cannot reach the host LAN or internet; adding this
   module doesn't require editing any other experiment's files.

Keep theory.md and references.md free of any copyrighted text — summarize
concepts in your own words, don't quote textbooks.
```

---

7. Integration & distribution

- One shared repo, branch `group-EXP-2` (confirm the real EXP ID with faculty — the ARP Spoofing row wasn't assigned one explicitly in the allotment sheet).
- Each member works in their own subfolder; merge conflicts should be rare since ownership is file-path-separated.
- Before opening the PR into `main`: run `docker compose up -d --build` from a clean clone on a machine nobody has set up before — this is the real test of "distributable."
- PR must include the filled README.md (Module 4) so the integration team can plug it into the main navigation without asking you questions.

8. Safety checklist (carried over from the attack-simulation guideline)

- `arp-lab-net` is `internal: true` — no route out to the host LAN or internet.
- Memory/CPU limits set on every service.
- No `docker exec` target ever gets unrestricted host-shell access — only the specific scripted commands in Section 3.
- Stop Lab and Reset Lab must fully remove containers, the network, and any volumes.
- Nothing runs arbitrary student-submitted code on the host.

9. Suggested schedule (deadline 20/10, eval 26–31/10)

Date	Milestone
16/10	Group registered; EXP ID confirmed with faculty
12–13/10	Each module owner has their prompt output running standalone
15/10	Module 1 + 2 integrated: backend can actually trigger a real poison via Docker
17/10	Module 3 wired to live backend; Module 4's quiz and theory dropped in